Homework 5

Code

```
import numpy as np
import matplotlib.pyplot as plt

val = 4 * np.log(2) + 2

def f(x):
    return x**2 * np.log(x)

def fd(h):
    return (f(2+h) - f(2))/h

#for part (b)
ns = [n for n in range(1, 16)]
hs = [10**(-n) for n in ns]
errors = [abs(fd(h) - val) for h in hs]

best_n = ns[np.argmin(errors)]

#for part (c)
better_hs = [10**(-best_n - 1 + 0.01*m) for m in range(0, 200)]
better_errors = [abs(fd(h) - val) for h in better_hs]

#Ploting -- Note switch out the first two plt.loglog comments
#           to see a specific graph

#plt.loglog(hs, errors)
plt.loglog(better_hs, better_errors)
plt.xlabel('h')
plt.ylabel('Absolute Error')
plt.title('Forward Difference Error vs h (log-log)')
plt.grid(True, which='both')
plt.show()
```

Graphs

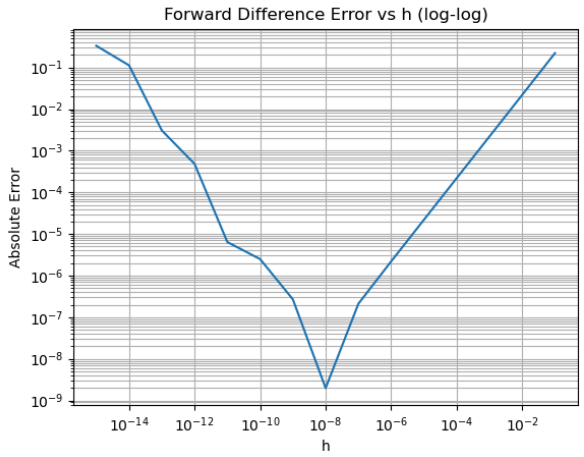

Figure 1: Part (b)

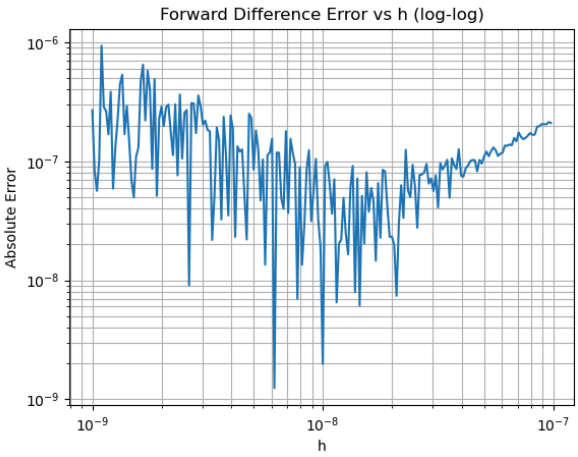

Figure 2: Part (c)

Discussion

Part (b) — We can see that for larger $h \sim 10^{-2}$, as h decreases toward 10^{-8}, the absolute error decreases. This makes sense since the error is $O(h)$ and the log log plot looks linear there. After we reach the minima, namely 10^{-8}, the error increases again. This is likely due to the round off error for very small h.

Part (c) — A similar discussion as in part (b) can be applied here. On the right of $h = 10^{-8}$ it can be seen that the error is decreasing roughly in a linear scale, yet becoming more oscillatory as h decreases. After the 10^{-8} mark the oscillatory behaviour becomes drastic which directly reflects round off errors for smaller and smaller h.